

Unit 1 Practice Test

Practice 1.2

Write a program to prompt the user for his grade on a test, and inform him if he passed:

Example

User input is underlined.

```
What was your grade on the last test? 92  
You passed the test.
```

Another example

```
What was your grade on the last test? 59  
You failed the test.
```

Three Functions

Your program needs to have three functions: one to prompt the user for his grade, one to display the "passed" message, and one to display the "failed" message.

Assignment

Please copy into your home directory the course template from:

```
/home/cs124/tests/templateTest1.cpp
```

Use it to create the file for your test. Please:

- Write the program.
- Compile it.
- Run it to make sure it gives you the output you expect.
- Run the style checker and fix any style errors.
- Run the test bed and make sure that the output is exactly as expected:

```
testBed cs124/practice12 practice12.cpp
```

A sample solution is on:

```
/home/cs124/tests/practice12.cpp
```

Continued on the next page

Grading for Test1

Sample grading criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Copy template 10%	Template is copied			Need a hint	Something other than the standard template is used
Compile 20%	No compile errors or warnings	One warning	One error	Two errors	Three or more compile errors
Modularization 20%	Functions used effectively in the program	No bugs exist in the declaration or use of functions	One bug exists in the function	Two bugs	All the code exists in one function
Conditional 10%	The conditional is both elegant and efficient	A conditional exists that determines if the grade is sufficient	A bug exists in the conditional	Elements of the solution are present	No attempt was made at the solution
I/O 20%	Zero test bed errors	Looks the same on screen, but minor test bed errors	One major test bed error	One or more tests pass test bed	Program input and output do not resemble the problem
Programming Style 20%	Well commented, meaningful variable names, effective use of blank lines	Zero style checker errors	One or two minor style checker errors	Code is readable, but serious style infractions	No evidence of the principles of "elements of style" in the program

Unit 1 Project : Monthly Budget

Our first project will be to write a program to manage a user's personal finances for a month. This program will ask the user for various pieces of financial information then will then display a report of whether the user is on target to meet his or her financial goals.

This project will be done in three phases:

- Project 02 : Prompt the user for input and display the input back in a table
- Project 03 : Split the program into separate functions and do some of the budget calculations
- Project 04 : Determine the tax burden

Interface Design

The following is an example run of the program. An example of input is underlined.

```
This program keeps track of your monthly budget
Please enter the following:
```

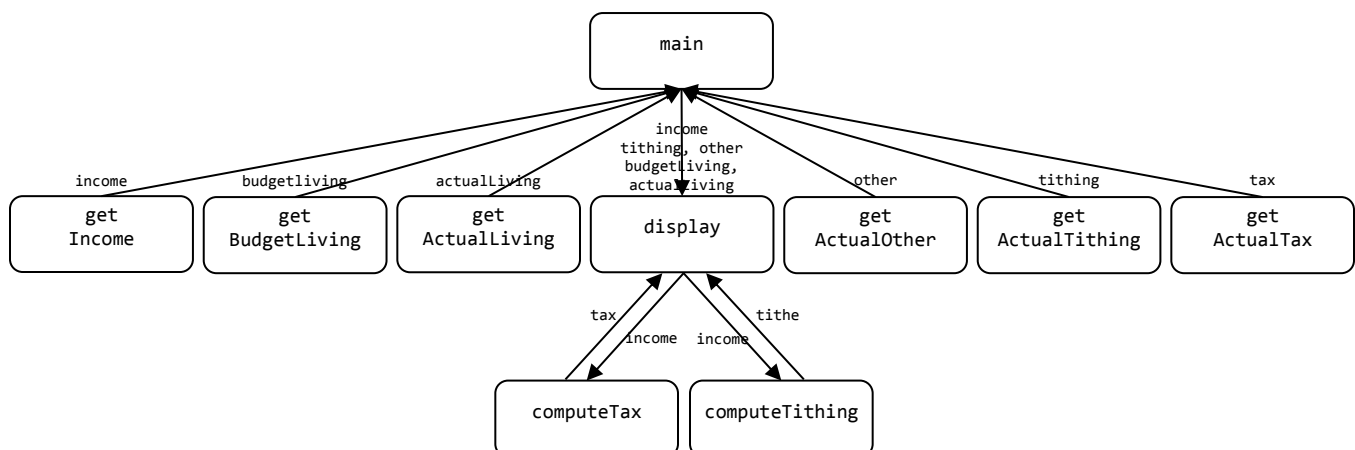
```
Your monthly income: 1000.00
Your budgeted living expenses: 650.00
Your actual living expenses: 700.00
Your actual taxes withheld: 100.00
Your actual tithe offerings: 120.00
Your actual other expenses: 150.00
```

```
The following is a report on your monthly expenses
```

Item	Budget	Actual
Income	\$ 1000.00	\$ 1000.00
Taxes	\$ 100.00	\$ 100.00
Tithing	\$ 100.00	\$ 120.00
Living	\$ 650.00	\$ 700.00
Other	\$ 150.00	\$ 150.00
Difference	\$ 0.00	\$ -70.00

Structure Chart

You may choose to use the following functions as part of your design:



Algorithms

main()

Main is the function that signifies the beginning of execution. Typically `main()` does not do anything; it just calls other functions to get the job done. You can think of `main()` as a delegator. For this program, `main()` will call the get functions, call the `display()` function, and hold the values that the user has input. The pseudocode (described in [chapter 2.2](#)) for `main()` is:

```
main
  PUT greeting on the screen

  income ← getIncome()
  budgetLiving ← getBudgetLiving()
  actualLiving ← getActualLiving()
  actualTax ← getActualTax()
  actualTithing ← getActualTithing()
  actualOther ← getActualOther()

  display(income, budgetLiving, actualTax, actualTithing, actualLiving, actualOther)
end
```

getIncome()

The purpose of `getIncome()` is to prompt the user for his income and return the value to `main()`:

```
getIncome
  PROMPT for income
  GET income
  RETURN income
end
```

The pseudocode for the other get functions is the same. Note that all input from the users is gathered in the “get” functions. Also note that there is a tab before the “Your monthly income:”

display()

Display takes the input gathered from the other modules and puts it on the screen in an easy to read format. Display formats the output, displays some of the data to the user, and calls other functions to display the rest. The pseudocode for `display()` is the following:

```
display ( income, budgetLiving, actualTax, actualTithing, actualLiving, actualOther)
  SET budgetTax ← computeTax(income)
  SET budgetTithing ← computeTithing(income)
  SET budgetOther ← income – budgetTax – budgetTithing – budgetLiving
  SET actualDifference ← income – actualTax – actualTithing – actualLiving – actualOther
  SET budgetDifference ← 0

  PUT row header on the screen

  PUT income
  PUT budgetTax, actualTax,
  PUT budgetTithing, actualTithing,
  PUT budgetLiving, actualLiving,
  PUT budgetOther, actualOther,
  PUT budgetDifference, actualDifference
end
```

A few hints:

- A tab used for most of the indentations.
- The difference row is the difference between income and expense. Note that the difference for Budget should be zero: you plan on balancing your budget!
- Please follow the design presented in the Structure Chart (described in [chapter 2.0](#)) for your project. You may choose to add functions to increase code clarity (such as the `budgetOther` and `actualDifference` computation in `display()`).

`computeTithing()`

The purpose of `computeTithing()` is to determine amount that is required to be a full tithe payer. This does not include fast offerings and other offerings. The pseudocode for `computeTithing()` is:

And after that, those who have thus been tithed shall pay one-tenth of all their interest annually; and this shall be a standard law unto them forever, for my holy priesthood, saith the Lord. D&C 119:4

`computeTax()`

In order to determine the tax burden, it is necessary to project the monthly income to yearly income, compute the tax, and reduce that amount back to a monthly amount. In each case, it is necessary to determine the tax bracket of the individual and to then apply the appropriate formula. The tax brackets for the 2006 year are:

If taxable income is over--	But not over--	The tax is:
\$0	\$15,100	10% of the amount over \$0
\$15,100	\$61,300	\$1,510.00 plus 15% of the amount over 15,100
\$61,300	\$123,700	\$8,440.00 plus 25% of the amount over 61,300
\$123,700	\$188,450	\$24,040.00 plus 28% of the amount over 123,700
\$188,450	\$336,550	\$42,170.00 plus 33% of the amount over 188,450
\$336,550	no limit	\$91,043.00 plus 35% of the amount over 336,550

The pseudocode for `computeTax()` is the following:

```

computeTax (monthlyIncome)
    yearlyIncome ← monthlyIncome * 12

    if ($0 ≤ yearlyIncome < $15,100)
        yearlyTax ← yearlyIncome * 0.10
    if ($15,100 ≤ yearlyIncome < $61,300)
        yearlyTax ← $1,510 + 0.15 *(yearlyIncome - $15,100)
    if ($61,300 ≤ yearlyIncome < $123,700)
        yearlyTax ← $8,440 + 0.25 *(yearlyIncome - $61,300)
    if ($123,700 ≤ yearlyIncome < $188,450)
        yearlyTax ← $24,040 + 0.28 *(yearlyIncome - $123,700)
    if ($188,450 ≤ yearlyIncome < $336,550)
        yearlyTax ← $42,170 + 0.33 *(yearlyIncome - $188,450)
    if ($336,550 ≤ yearlyIncome)
        yearlyTax ← $91,043 + 0.35 *(yearlyIncome - $336,550)

    monthlyTax ← yearlyTax / 12

    return monthlyTax
end

```

Note that this algorithm is vastly oversimplified because it does not take into account deductions and other credits. Please do not use this algorithm to compute your actual tax burden!

Project 02

The first submission point due at the end of Week 02 is to prompt the user for input and display the budget back on the screen:

```
This program keeps track of your monthly budget
Please enter the following:
```

```
Your monthly income: 1000.00
Your budgeted living expenses: 650.00
Your actual living expenses: 700.00
Your actual taxes withheld: 100.00
Your actual tithe offerings: 120.00
Your actual other expenses: 150.00
```

```
The following is a report on your monthly expenses
```

Item	Budget	Actual
Income	\$ 1000.00	\$ 1000.00
Taxes	\$ 0.00	\$ 100.00
Tithing	\$ 0.00	\$ 120.00
Living	\$ 650.00	\$ 700.00
Other	\$ 0.00	\$ 150.00
Difference	\$ 0.00	\$ 0.00

A few hints:

- A tab used for most of the indentations and nowhere else.
- There are 15 '='s under Income, Budget, and Actual.
- The user's monthly income is used both for the Budget value and for the Actual value
- The Budget value for Taxes, Tithing, and Other will always be zero. Also the Difference, both Budget and Actual, will be zero. We will compute these in the next two parts of this project.

To complete this project, please do the following:

1. Copy the course template from:

```
/home/cs124/template.cpp
```

2. Write each function. Test them individually before “hooking them up” to the rest of the program.
3. Compile and run the program to ensure that it works as you expect:

```
g++ project02.cpp
```

4. Test the program with testbed and fix all the errors:

```
testBed cs124/project02 project1.cpp
```

5. Run the style checker and fix all the errors:

```
styleChecker project02.cpp
```

6. Submit it with “Project 02, Monthly Budget” in the program header:

```
submit project02.cpp
```

Project 03

This second part of the Monthly Budget project will be to divide the program into functions and perform some of the simple calculations:

```
This program keeps track of your monthly budget
Please enter the following:
```

```
Your monthly income: 1000.00
Your budgeted living expenses: 650.00
Your actual living expenses: 700.00
Your actual taxes withheld: 100.00
Your actual tithe offerings: 120.00
Your actual other expenses: 150.00
```

```
The following is a report on your monthly expenses
```

```
=====
Income      $    1000.00   $    1000.00
Taxes       $      0.00   $     100.00
Tithing     $     100.00   $     120.00
Living      $     650.00   $     700.00
Other       $     250.00   $     150.00
=====
Difference  $      0.00   $     -70.00
```

Notice how many of the values that were previously 0.00 now are computed. You will also need to calculate the values for Tithing, Budget Other, and Actual Difference. You can find the formula for these calculations earlier in the project description.

To complete this project, please do the following:

1. Start from the work you did in Project 02.
2. Fix any defects.
3. Write each function. Test them individually before "hooking them up" to the rest of the program.
4. Compile and run the program to ensure that it works as you expect:

```
g++ project03.cpp
```

5. Test the program with testbed and fix all the errors:

```
testBed cs124/project03 project03.cpp
```

6. Run the style checker and fix all the errors:

```
styleChecker project03.cpp
```

7. Submit it with "Project 03, Monthly Budget" in the program header:

```
submit project03.cpp
```

An executable version of the project is available at:

```
/home/cs124/projects/prj03.out
```

Project 04

This final part of the Monthly Budget project will be to add the compute tax component.

This program keeps track of your monthly budget
Please enter the following:

Your monthly income: 1000.00
Your budgeted living expenses: 650.00
Your actual living expenses: 700.00
Your actual taxes withheld: 100.00
Your actual tithe offerings: 120.00
Your actual other expenses: 150.00

The following is a report on your monthly expenses

Income	\$	1000.00	\$	1000.00
Taxes	\$	100.00	\$	100.00
Tithing	\$	100.00	\$	120.00
Living	\$	650.00	\$	700.00
Other	\$	150.00	\$	150.00
=====				
Difference	\$	0.00	\$	-70.00

Notice how the taxes were computed to \$100.00 where in Project 02 they were set to 0.00.

To complete this project, please do the following:

1. Start from the work you did in Project 03.
2. Fix any defects and implement the `computeTax()` function.
3. Compile and run the program to ensure that it works as you expect.:

```
g++ project04.cpp
```

4. Test the program with testbed and fix all the errors:

```
testBed cs124/project04 project04.cpp
```

5. Run the style checker and fix all the errors:

```
styleChecker project04.cpp
```

6. Submit it with "Project 04, Monthly Budget" in the program header:

```
submit project04.cpp
```

An executable version of the project is available at:

```
/home/cs124/projects/prj04.out
```